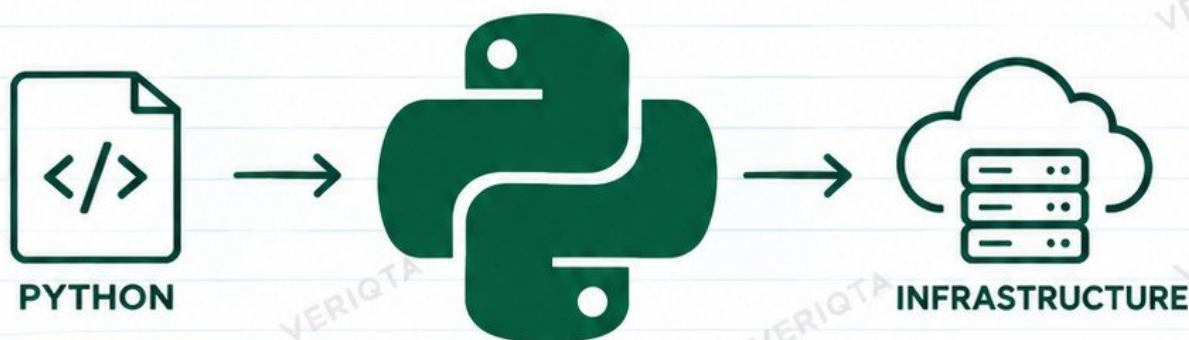


PYTHON FOR DEVOPS AUTOMATION HANDBOOK

From Python Fundamentals to
Production Infrastructure Automation



LINUX



AWS



DOCKER



KUBERNETES



CI/CD



MONITORING



WHAT YOU WILL MASTER

Python Basics | Scripting | Files | JSON/YAML | Linux | subprocess
APIs | AWS | Docker | Kubernetes | CI/CD | Observability |
Production Automation

AUTOMATE . SIMPLIFY . DEPLOY



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

WHY DEVOPS ENGINEERS USE PYTHON

Python is the automation glue of DevOps.
It helps you build reliable, repeatable and scalable solutions.

1. WHERE PYTHON FITS IN DEVOPS



2. PYTHON vs BASH

PYTHON	BASH
 High-level language	 Shell scripting language
 Better for complex logic & large projects	 Better for simple one-liners
 Strong libraries and modules	 Limited built-in libraries
 Better error handling and debugging	 Harder to debug at scale
 Easier to integrate APIs & systems	 Hard to work with APIs & JSON
 More maintainable and reusable	 Harder to scale and maintain

3. TASKS WORTH AUTOMATING

- ✓ Server provisioning and configuration
- ✓ User and permission management
- ✓ Application deployments and upgrades
- ✓ Log collection and analysis
- ✓ Backup, cleanup and housekeeping
- ✓ Monitoring, alerting and notifications
- ✓ Security scans and compliance checks
- ✓ Cloud resource management and cost reports
- ✓ CI/CD pipeline operations

4. REPETITIVE WORK = HUMAN ERROR

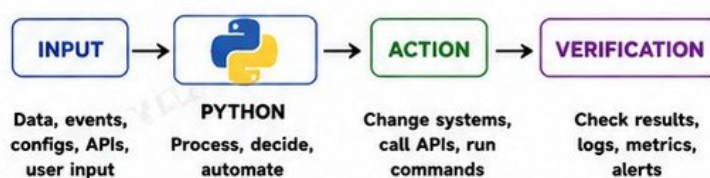


Automation reduces human error, improves speed, consistency and reliability.

5. PYTHON POWERS



6. MENTAL MODEL



JUNIOR ENGINEER TIP

AUTOMATE WHAT YOU UNDERSTAND FIRST.

Start small, test thoroughly, and build trust in your automation.
BAD AUTOMATION IS WORSE THAN NO AUTOMATION.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

PYTHON SETUP FOR DEVOPS ENGINEERS



1. INSTALLING PYTHON 3



Most Linux distributions

```
sudo apt update && sudo apt install -y python3-pip
```



macOS (with Homebrew)

```
brew install python3
```



Windows

Download from python.org and run the installer.

2. CHECK PYTHON VERSION



Verify Python 3 is installed.

```
python3 --version
```

Example output:

```
Python 3.11.6
```

3. RUNNING .PY FILES



app.py

Create a file (app.py):

```
print("Hello DevOps!")  
print("Python is ready 🚀")
```

Run it:

```
python3 app.py
```

4. INTERACTIVE PYTHON



Start the interactive shell.

```
python3
```

Example:

```
>>> 2 + 2  
4  
>>> exit()
```

5. PIP - PYTHON PACKAGE INSTALLER



pip

pip installs libraries and tools.

Check pip:

```
pip3 --version
```

Install a package:

```
pip3 install requests
```

6. VIRTUAL ENVIRONMENTS



Virtual environments isolate dependencies for each project.

Create a virtual environment:

```
python3 -m venv .venv
```

7. ACTIVATING ENVIRONMENTS



Linux / macOS

```
source .venv/bin/activate
```



Windows (CMD)

```
.venv\Scripts\activate.bat
```



Windows (PowerShell)

```
.venv\Scripts\Activate.ps1
```



You'll see (.venv) in your terminal prompt when active.

8. REQUIREMENTS.TXT



requirements.txt

List project dependencies in a file.

Example requirements.txt:

```
requests==2.32.3  
boto3==1.34.95  
pyyaml==6.0.1
```

Install everything:

```
pip3 install -r requirements.txt
```

9. WHY DEPENDENCY ISOLATION MATTERS



AVOID CONFLICTS

Different projects may need different versions of the same library.



REPRODUCIBLE ENVIRONMENTS

Anyone can recreate the same environment using requirements.txt.



SAFE UPDATES

Update packages in one project without breaking others.



CLEANER SYSTEMS

Keeps your global Python environment clean and stable.



PRODUCTION READINESS

Isolation reduces risk and improves reliability in automation.



JUNIOR ENGINEER TIP

Always use a virtual environment for every Python project. It keeps your automation stable, portable and production-ready.

PYTHON FUNDAMENTALS YOU ACTUALLY NEED



THE BUILDING BLOCKS OF EVERY PYTHON DEVOPS SCRIPT

1. VARIABLES

Store values in a name.

```
name = "web-server-01"
ip = "10.0.0.10"
port = 22
enabled = True
```

2. STRINGS

Text values.

```
greeting = "Hello DevOps!"
user = 'admin'
message = "Python is awesome"
multiline = """
This is
a multiline
string"""
```

3. INTEGERS & FLOATS

Numbers.

```
count = 10
cpu_cores = 4
price = 19.99
pi = 3.14159
```

4. BOOLEANS

True or False values.

```
is_running = True
is_healthy = False
maintenance = False
```

5. LISTS

Ordered, changeable collection.

```
servers = ["web-01", "web-02", "db-01"]
ports = [22, 80, 443]
mixed = ["admin", 10, True]
servers.append("web-03")
first_server = servers[0]
```

6. DICTIONARIES

Key-value pairs.

```
server = {
    "name": "web-01",
    "ip": "10.0.0.10",
    "role": "web",
    "port": 80,
    "active": True
}
server["ip"]
server.get("role")
```

7. TUPLES

Ordered, unchangeable collection.

```
coords = (10.0, 20.0)
version = (3, 11, 6)
first = coords[0]
```

8. SETS

Unordered collection of unique items.

```
unique_ports = {22, 80, 443}
unique_ports.add(8080)
unique_ports.remove(22)
22 in unique_ports # False
80 in unique_ports # True
```

9. TYPE CONVERSION

Convert between data types.

```
str_num = "100"
int_num = int(str_num) # 100
float_num = float(int_num) # 100.0
str_float = str(float_num) # '100.0'
bool_val = bool(1) # True
bool_empty = bool("") # False
```

10. PRINT()

Display output.

```
print("Hello DevOps!")
print("Server:", name, "IP:", ip)
print(f"Server {name} is running on {ip}:{port}")
```

11. PRACTICAL EXAMPLE: REPRESENTING SERVER INFORMATION

Using the fundamentals to model a real server.

```
server = {
    "name": "web-01",
    "ip": "10.0.0.10",
    "port": 80,
    "cpu": 2,
    "memory_gb": 4.0,
    "roles": ["web", "nginx", "frontend"],
    "tags": {"env": "prod", "team": "platform"},
    "active": True
}
```

```
print(f"Server: {server['name']}")
print(f"IP: {server['ip']}")
print(f"Roles: {', '.join(server['roles'])}")
print(f"Env: {server['tags']['env']}")
print(f"Active: {server['active']}")

# Access nested values
env = server['tags']['env']
is_active = server.get('active', False)
```

OUTPUT

```
Server: web-01
IP: 10.0.0.10
Roles: web, nginx,
frontend
Env: prod
Active: True
```



JUNIOR ENGINEER TIP

Master these building blocks.

They are all you need to automate real infrastructure.

CONDITIONS, LOOPS, AND AUTOMATION LOGIC



TURNING DECISIONS INTO ACTIONS

1. IF, ELIF, ELSE

Make decisions in your automation.

```
status = "running"

if status == "running":
    print("Service is running")

elif status == "stopped":
    print("Service is stopped")

else:
    print("Unknown status")
```

2. COMPARISON OPERATORS

Compare values.

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
in	Membership
not in	Not in membership

3. BOOLEAN LOGIC

Combine conditions.

and True if both conditions are True
or True if at least one is True
not Reverse the result

Example:

```
cpu = 65
memory = 40
if cpu > 80 and memory > 70:
    print("High load alert")
```

4. FOR LOOPS

Iterate over a sequence.

```
servers = ["web-01", "web-02",
           "db-01", "cache-01"]

for server in servers:
    print(f"Checking {server}")
```

5. WHILE LOOPS

Repeat while a condition is True.

```
count = 0

while count < 3:
    print(f"Attempt {count + 1}")
    count += 1
```

6. BREAK AND CONTINUE

Control loop flow.

```
for server in servers:
    if server == "db-01":
        continue # Skip db-01

    if server == "cache-01":
        break # Stop loop

    print(f"Processing {server}")
```

7. LOOPING THROUGH SERVERS

Example: check connectivity to servers.

```
servers = ["web-01", "web-02", "db-01"]

for server in servers:
    # Simulate a health check result
    is_up = server != "db-01"

    if is_up:
        print(f"{server}: ✅ UP")
    else:
        print(f"{server}: ❌ DOWN")
```



8. CHECKING SERVICE STATES

Example: verify service status on each server.

```
services = ["nginx", "docker", "ssh"]

for server in servers:
    print(f"\n{server}")
    for svc in services:
        # Simulated status
        status = "running" if svc != "docker" else "stopped"

        if status == "running":
            print(f"    {svc}: ✅ {status}")
        else:
            print(f"    {svc}: ❌ {status}")
```



9. TURNING OPERATIONAL RULES INTO CODE

Example Rule:

"Restart the service if it is not running and CPU usage is below 80%."



Python Implementation:

```
service_status = "stopped" # from check
cpu_usage = 55 # from monitoring

if service_status != "running" and cpu_usage < 80:
    print("Restarting service...")
    # restart_service()
else:
    print("No action required.")
```



JUNIOR ENGINEER TIP

Automate what you understand first.

Start small, test often, and trust the logic before scaling.

FUNCTIONS AND REUSABLE AUTOMATION



WRITE ONCE. USE MANY TIMES.

1. WHY FUNCTIONS MATTER

- ✓ Avoid repeating code.
- ✓ Make scripts easier to read.
- ✓ Easier to test and debug.
- ✓ Encourage modular, maintainable automation.
- ✓ Reuse logic across projects.

★ Functions turn your ideas into reusable building blocks.

2. DEFINING FUNCTIONS

Use the `def` keyword.

```
def greet(name):  
    message = f"Hello, {name}!"  
    print(message)
```

Call the function:

```
greet("DevOps Engineer")
```

3. PARAMETERS & ARGUMENTS

Parameters are variables in the function definition.
Arguments are the actual values you pass.

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result) # 30
```

4. RETURN VALUES

Return a value using `return`.

```
def get_status():  
    return "OK"  
  
status = get_status()  
print(status) # OK
```

5. DEFAULT ARGUMENTS

Provide default values.

```
def check_port(host, port=80):  
    print(f"Checking {host}:{port}")  
  
check_port("web-01") # Uses 80  
check_port("web-01", 443) # Uses 443
```

6. SCOPE BASICS

Variables inside a function are local by default.

```
message = "Global"  
  
def show():  
    message = "Local"  
    print(message)  
  
show()  
print(message) # Global
```

SCOPE RULE

LOCAL
Inside Function



GLOBAL
Outside Function

7. BREAKING LARGE SCRIPTS INTO FUNCTIONS

Split complex automation into clear, focused functions.



- ✓ Read Config
- ✓ Check Hosts
- ✓ Check Services
- ✓ Generate Report
- ✓ Send Alert

Instead of one long script...

```
# Thousands of lines...  
.....  
.....  
.....
```



Break into small, reusable functions.

```
load_config()  
check_host()  
check_services()  
generate_report()  
send_alert()  
main()
```



JUNIOR ENGINEER TIP

Small functions are easier to test, reuse and trust.
One job per function.

8. REUSABLE HEALTH-CHECK EXAMPLE

Functions help you reuse logic across environments and projects.

```
import socket  
  
def is_host_up(host, port=22, timeout=3):  
    try:  
        with socket.create_connection((host, port), timeout):  
            return True  
    except:  
        return False  
  
def check_hosts(hosts):  
    results = {}  
    for host in hosts:  
        results[host] = is_host_up(host)  
    return results
```

```
def print_report(results):  
    for host, status in results.items():  
        icon = "✅ UP" if status else "❌ DOWN"  
        print(f"{host}: {icon}")  
  
def main():  
    servers = ["web-01", "web-02", "db-01"]  
    results = check_hosts(servers)  
    print_report(results)  
  
if __name__ == "__main__":  
    main()
```

SAMPLE OUTPUT

```
web-01: ✅ UP  
web-02: ✅ UP  
db-01: ❌ DOWN
```

★ This logic can be reused for any port, any host, any environment.



JUNIOR ENGINEER TIP

Build small functions. Combine them into powerful automation.
Reusable code saves time and prevents mistakes.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



FILES, PATHS, AND CONFIGURATION AUTOMATION



AUTOMATE FILE OPERATIONS. MANAGE CONFIGS SAFELY.

1. READING FILES

Read the entire file or line by line.



```
with open("app.log", "r") as f:
    content = f.read()
    print(content)

# Read line by line
with open("app.log", "r") as f:
    for line in f:
        print(line.strip())
```

2. WRITING FILES

Create or overwrite files.



```
with open("output.txt", "w") as f:
    f.write("Hello DevOps\n")
    f.write("Automation is powerful\n")

# Append to a file
with open("output.txt", "a") as f:
    f.write("More data\n")
```

3. WITH OPEN()

Always use `with open()` for automatic cleanup.

- ✓ Automatically closes the file
- ✓ Prevents resource leaks
- ✓ Cleaner and safer code

4. PATHLIB - MODERN PATH HANDLING

Simple and cross-platform path operations.



pathlib

```
from pathlib import Path
path = Path("/etc/nginx/nginx.conf")
print(path.name)      # nginx.conf
print(path.parent)    # /etc/nginx
print(path.stem)      # nginx
print(path.suffix)     # .conf
print(path.exists())  # True/False
print(path.is_file()) # True/False
print(path.is_dir())  # True/False
```

5. CHECK IF FILE/DIRECTORY EXISTS

Validate before operations.

```
from pathlib import Path
file_path = Path("/etc/hosts")
dir_path = Path("/etc/nginx")
if file_path.exists():
    print("File exists")
else:
    print("File not found")

if dir_path.exists():
    print("Directory exists")
else:
    print("Directory not found")
```



6. CREATING DIRECTORIES

Create directories if they don't exist.

```
from pathlib import Path
Path("/opt/myapp").mkdir()

# Create parents if missing
Path("/opt/myapp/logs").mkdir(
    parents=True,
    exist_ok=True
)
```

- ★ `exist_ok=True` prevents errors if the directory already exists.

7. READING CONFIGURATION

Load configuration from files.

YAML Example

```
import yaml
with open("config.yaml", "r") as f:
    config = yaml.safe_load(f)

print(config["app"]["port"])
print(config["db"]["host"])
```

JSON Example

```
import json
with open("config.json", "r") as f:
    config = json.load(f)

print(config["app"]["name"])
print(config["features"])
```

- ★ Keep configuration separate from code. It makes automation flexible and environment-ready.

8. SAFELY MODIFYING FILES

Read, modify, and write safely.

```
from pathlib import Path
path = Path("/etc/myapp/app.conf")
lines = path.read_text().splitlines()

new_lines = []
for line in lines:
    if line.startswith("port="):
        new_lines.append("port=8080")
    else:
        new_lines.append(line)

path.write_text("\n".join(new_lines) + "\n")
```



9. BACKING UP BEFORE CHANGES

Always create a backup first.

```
import shutil
from pathlib import Path
src = Path("/etc/myapp/app.conf")
backup = src.with_suffix(
    src.suffix + ".bak")
shutil.copy2(src, backup)
print(f"Backup created: {backup}")
```

- ★ Backups are your safety net. Never modify critical files without one.

10. AUTOMATING LINUX CONFIGURATION TASKS

A simple example: ensure a line exists in a config file.

```
from pathlib import Path

def ensure_line(file_path, line_to_ensure):
    path = Path(file_path)
    backup = path.with_suffix(path.suffix + ".bak")
    shutil.copy2(path, backup) # Backup

    lines = path.read_text().splitlines()
    if line_to_ensure not in lines:
        lines.append(line_to_ensure)
        path.write_text("\n".join(lines) + "\n")
        print(f"Added line: {line_to_ensure}")
    else:
        print("Line already present, no changes.")
```

Example Usage:

```
ensure_line("/etc/ssh/sshd_config", "ClientAliveInterval 300")
```

- ✓ Ensures a configuration rule exists
- ✓ Creates backup automatically
- ✓ Safe, repeatable, and idempotent
- ✓ Perfect for automation across servers

COMMON FILE MODES

```
"r"   Read
"w"   Write (overwrite)
"a"   Append
"r+"  Read and write
"x"   Create new file
```

BEST PRACTICES

- ✓ Always validate paths and inputs
- ✓ Use `pathlib` for path operations
- ✓ Back up before modifying critical files
- ✓ Keep configs outside your scripts
- ✓ Write idempotent automation



JUNIOR ENGINEER TIP

Automate configuration tasks carefully.

Test on a lab environment before running on production systems.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

AUTOMATE SMARTER. BUILD RELIABLE SYSTEMS. SHIP WITH CONFIDENCE.

WORKING WITH JSON, YAML, AND ENVIRONMENT VARIABLES

1 WHY DEVOPS USES STRUCTURED DATA

- Infrastructure, tools, and services exchange data in structured formats.
- JSON and YAML are human-readable, machine-parseable, and widely used.
- Environment variables keep configuration dynamic and secure.
- This makes automation reliable, portable, and scalable.



JSON



YAML



ENV VARS

2 PYTHON JSON

READING JSON

```
import json
with open('config.json') as f:
    data = json.load(f)
print(data['app']['name']) # myapp
```

WRITING JSON

```
import json
data = {'app': {'name': 'myapp', 'replicas': 3}}
with open('output.json', 'w') as f:
    json.dump(data, f, indent=2)
```

3 YAML WITH PYYAML

READING YAML

```
import yaml
with open('config.yaml') as f:
    data = yaml.safe_load(f)
print(data['database']['host']) # db.prod.local
```

WRITING YAML

```
import yaml
data = {'app': {'name': 'myapp', 'replicas': 3}}
with open('output.yaml', 'w') as f:
    yaml.safe_dump(data, f, sort_keys=False)
```

4 ENVIRONMENT VARIABLES

Environment variables store configuration outside your code. Python accesses them using `os.environ`.

```
import os
db_user = os.environ['DB_USER']
db_pass = os.environ['DB_PASS']
debug = os.environ.get('DEBUG', 'false')
print(db_user)
```



TIP: Use `os.environ.get()` with defaults to avoid `KeyError` if a variable is missing.

5 CONFIGURATION VS SECRETS

CONFIGURATION (NOT SECRET)

- Example: APP_NAME, LOG_LEVEL
- Safe to store in config files, YAML, or environment variables
- These values can be public

SECRETS (SENSITIVE)

- Example: DB_PASSWORD, API_KEY
- Do NOT store in code or Git
- Use environment variables, secret managers, or CI/CD secret storage



NEVER HARDCODE CREDENTIALS. EVER. NOT IN CODE. NOT IN SCRIPTS. NOT IN CONFIG FILES.

6 PARSING KUBERNETES AND CI/CD CONFIG

Python reads Kubernetes and CI/CD configs (JSON or YAML) to automate tasks.

EXAMPLE: KUBERNETES POD INFO (YAML)

```
import yaml
with open('pod.yaml') as f:
    pod = yaml.safe_load(f)
print(pod['metadata']['name'])
```

EXAMPLE: GITHUB ACTIONS (YAML)

```
with open('.github/workflows/deploy.yml') as f:
    workflow = yaml.safe_load(f)
print(workflow['jobs'].keys())
```

7 GOLDEN RULES

- ✓ Use structured data (JSON/YAML) for reliable automation.
- ✓ Keep configuration separate from code.
- ✓ Use environment variables for dynamic values.
- ✓ Protect secrets using secure storage.
- ✓ Validate data before using it.
- ✓ Never hardcode credentials.

COMMON FORMATS YOU WILL SEE



JSON



YAML



ENV VARS



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LINUX AND SYSTEM AUTOMATION WITH PYTHON

1 PYTHON MODULES FOR LINUX & SYSTEM TASKS

	os	Interact with the operating system and environment variables.
	sys	Access Python runtime info, arguments, and exit.
	shutil	High-level file operations like copy, move, delete.
	platform	Get system, OS, kernel, and machine information.
	psutil	System monitoring: CPU, memory, disk, processes, network, and more.

2 WHAT WE CAN AUTOMATE

 DISK USAGE	 CPU USAGE	 MEMORY CHECKS
 FILE OPERATIONS	 PROCESS INSPECTION	 HOST INFORMATION

3 EXAMPLE: DISK, CPU, MEMORY, AND HOST INFO



DISK USAGE

```
import shutil
usage = shutil.disk_usage('/')
print(f"Disk Usage: {usage.percent}% used")
```



CPU USAGE

```
import psutil
print(f"CPU Usage: {psutil.cpu_percent(interval=1)}%")
```



MEMORY CHECK

```
mem = psutil.virtual_memory()
print(f"Memory Usage: {mem.percent}% used")
```



HOST INFORMATION

```
import platform, socket
print(f"OS: {platform.system()} {platform.release()}")
print(f"Hostname: {socket.gethostname()}")
print(f"IP Address: {socket.gethostbyname(socket.gethostname())}")
```



PROCESS INSPECTION

```
for proc in psutil.process_iter(['pid', 'name', 'cpu_percent']):
    proc.info['cpu_percent'] = proc.cpu_percent(interval=0.1)
    print(proc.info)
```

4 SIMPLE SERVER HEALTH CHECKER

```
import shutil, psutil, platform, socket, time

def health_check():
    health = {}

    # Disk
    disk = shutil.disk_usage('/')
    health['disk'] = disk.percent

    # CPU
    health['cpu'] = psutil.cpu_percent(interval=1)

    # Memory
    mem = psutil.virtual_memory()
    health['memory'] = mem.percent

    # Host Info
    health['host'] = {
        'hostname': socket.gethostname(),
        'ip': socket.gethostbyname(socket.gethostname()),
        'os': platform.system() + ' ' + platform.release()
    }

    return health

if __name__ == '__main__':
    status = health_check()
    for k, v in status.items():
        print(f"{k}: {v}")
```

SAMPLE OUTPUT

```
disk: 42.7
cpu: 13.3
memory: 61.2
host: {'hostname': 'web-01', 'ip': '10.0.1.15', 'os': 'Linux 5.15.0'}
```

5 JUNIOR ENGINEER TIP



Automate the boring. Monitor the important. Collect the right data, act with caution, and always log what your script does.



CAUTION

- Scripts run with your permissions. Be careful.
- Never delete or modify critical files blindly.
- Test on a safe environment first.

6 REQUIREMENT

Install psutil to unlock system monitoring power:

```
pip install psutil
```

➤ psutil works on Linux, macOS, and Windows.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

RUNNING LINUX COMMANDS SAFELY WITH `subprocess`

1 WHY PYTHON SOMETIMES NEEDS EXTERNAL COMMANDS



- Some tasks are easier or only possible using system commands.
- Interact with tools (git, kubectl, docker, jq, etc).
- Use existing CLI utilities.
- Access OS features not exposed in Python libraries.
- Bridge gaps when no Python API exists.

2 THE WORKHORSE: `subprocess.run()`

```
subprocess.run(command, capture_output=True, text=True)
```

command

List of command and arguments.

capture_output=True

Capture stdout and stderr.

text=True

Return output as string.

3 CAPTURING STDOUT AND STDERR

```
import subprocess

result = subprocess.run(['ls', '-l', '/tmp'],
                        capture_output=True, text=True)

# Standard Output (what the command prints)
print("STDOUT:")
print(result.stdout)

# Standard Error (errors and warnings)
print("STDERR:")
print(result.stderr)
```



If there is no output, stdout or stderr will be an empty string.

4 EXIT CODES

```
result = subprocess.run(['grep', 'root', '/etc/passwd'],
                        capture_output=True, text=True)

print("Exit Code:", result.returncode)
```

COMMON EXIT CODE MEANINGS

0	Success
1	General error / Command failed
2	Misuse of shell builtins (bash)
126	Command found but not executable
127	Command not found
>128	Terminated by signal (128 + signal number)

5 `check=True`: RAISE EXCEPTION ON FAILURE

```
try:
    subprocess.run(['false'], check=True)
except subprocess.CalledProcessError as e:
    print("Command failed!")
    print("Exit code:", e.returncode)
    print("stderr:", e.stderr)
```



Use `check=True` when failure must stop your script immediately.

6 TIMEOUTS

```
try:
    result = subprocess.run(['sleep', '10'],
                            timeout=3, capture_output=True,
                            text=True)
except subprocess.TimeoutExpired as e:
    print("Command timed out!")
    print("Partial output:", e.stdout)
```



Timeouts prevent your script from hanging forever on stuck commands.

7 AVOIDING `shell=True` UNNECESSARILY



LESS SAFE (Avoid if possible)

```
subprocess.run("rm -rf " + user_input,
               shell=True)
```



SAFER (Use a list of arguments)

```
subprocess.run(['rm', '-rf', target_path])
```



Passing a list is safer and avoids shell interpretation.

8 COMMAND INJECTION RISK



Using `shell=True` with untrusted input is dangerous.

Example of a vulnerable code:

```
subprocess.run(f"ping {user_ip}", shell=True)
```

If `user_ip = "8.8.8.8; rm -rf /"`



Always sanitize inputs or avoid `shell=True` when handling user input.

9 PYTHON API VS SHELL COMMAND



USE PYTHON API WHEN POSSIBLE

- ✓ More portable
- ✓ Easier error handling
- ✓ Structured data (not text parsing)
- ✓ Better performance
- ✓ Example: Instead of using `df` command, use `psutil.disk_usage('/')`

USE SHELL COMMAND WHEN

- ✓ No Python library exists
- ✓ You need a specific CLI feature
- ✓ Interacting with complex tools (kubectl, git, docker, etc.)
- ✓ Prototyping quickly



JUNIOR ENGINEER TIP

Use `subprocess.run()` safely. Capture outputs, check exit codes, set timeouts, and avoid `shell=True`. Automate with confidence, not risk.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

EXCEPTIONS, LOGGING, AND RELIABLE SCRIPTS

1 WHAT ARE EXCEPTIONS?



Exceptions are errors that occur during program execution. They disrupt the normal flow of your script.

Handling them prevents crashes and makes your automation reliable.

2 THE try → except → else → finally FLOW

```
try:
    result = 10 / int('0')
    print(f"Result: {result}")
except ZeroDivisionError as e:
    print(f"ERROR: {e}")
else:
    print("Succeed! No exception occurred.")
finally:
    print("This always runs.")
```

try: Code that might fail
except: Handle the exception
else: Runs only if no exception occurs
finally: Runs no matter what (cleanup)



Use finally to close files, release connections, or clean up resources.

3 RAISING EXCEPTIONS

Raise exceptions when something is wrong in your logic.

```
def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero is not allowed!")
    return a / b
```



Raising clear exceptions makes your code easier to debug and maintain.

4 PYTHON LOGGING BASICS

Logging records what happens in your script. It is better than print() for production.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

logger = logging.getLogger(__name__)
```

5 LOG LEVELS

DEBUG	Detailed diagnostic info
INFO	General operational info
WARNING	Something unexpected happened, but script continues
ERROR	Serious issue, operation failed
CRITICAL	Critical failure, needs immediate attention

6 USING LOGGING IN YOUR SCRIPTS

```
logger.info("Script started")
logger.warning("Disk usage is above 80%")
logger.error("Failed to connect to database")
logger.debug("Raw response: %s", response)
logger.info("Script completed successfully")
```

SAMPLE LOG OUTPUT

```
2025-05-20 10:15:30 INFO Script started
2025-05-20 10:15:31 WARNING Disk usage is above 80%
2025-05-20 10:15:31 ERROR Failed to connect to database
2025-05-20 10:15:31 INFO Script completed successfully
```

7 EXIT CODES

Exit codes tell the calling system whether your script succeeded or failed.

CODE	MEANING
0	Success
1	General Error
2	Invalid Input / Usage
3	Temporary Failure
4+	Custom Application Errors

```
import sys
try:
    # your code here
    sys.exit(0) # success
except ValueError as e:
    logger.error(f"Invalid input: {e}")
    sys.exit(2)
except Exception as e:
    logger.exception("Unexpected error")
    sys.exit(1)
```

Always return meaningful exit codes. They are critical in automation and CI/CD.

8 FAIL SAFELY, NOT SILENTLY

✗ BAD (SILENT FAILURE)

```
try:
    risky_operation()
except:
    pass # hides the error
```

- Hides problems
- Hard to troubleshoot
- Can cause data loss
- Leads to unreliable automation

✓ GOOD (FAIL SAFELY)

```
try:
    risky_operation()
except Exception as e:
    logger.error("Operation failed: %s", e)
    sys.exit(1)
```

- Logs the error
- Exits with failure code
- Easy to detect and alert
- Keeps your systems reliable

9 GOLDEN RULES FOR RELIABLE SCRIPTS



HANDLE EXCEPTIONS

Always expect failures and handle them.



LOG EVERYTHING

Log useful info at the right level.



USE EXIT CODES

Communicate success or failure clearly.



PROTECT SENSITIVE DATA

Never log secrets or private data.



FAIL EARLY AND SAFE

Stop on bad states, don't continue blindly.



@veriqta



veriqta



veriqta



veriqta



@veriqta

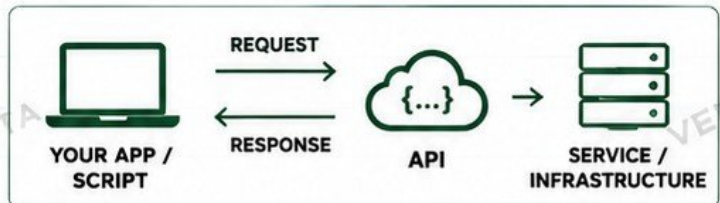


@veriqta

HTTP APIs FOR DEVOPS AUTOMATION

1 WHAT AN API IS

An API (Application Programming Interface) lets applications communicate with a system or service. It defines the rules for requests and responses.
You send a request → The API responds.



2 HTTP METHODS

GET	Retrieve data from the server.
POST	Create a new resource.
PUT	Replace an entire resource.
PATCH	Update part of a resource.
DELETE	Remove a resource.

3 STATUS CODES (COMMON)

200	OK – Request succeeded.
201	Created – Resource created.
204	No Content – Success, no data returned.
400	Bad Request – Invalid request.
401	Unauthorized – Auth required or failed.
403	Forbidden – No permission.
404	Not Found – Resource not found.
500	Internal Server Error – Server issue.

4 HEADERS

Headers provide metadata about the request or response.

COMMON HEADERS

Content-Type	Type of data (e.g. application/json)
Accept	What response format you expect
Authorization	Credentials or tokens
User-Agent	Identifies your client
X-Request-ID	Useful for tracing requests

5 AUTHENTICATION

- API Keys (in headers or query)
- Bearer Tokens (most common)
- Basic Auth (username:password)
- OAuth / OpenID (for user auth)
- Use HTTPS always
- Store secrets securely (env vars, vaults, secret managers)

6 JSON RESPONSES

APIs usually return JSON.

```
{
  "id": "i-1234567890abcdef0",
  "name": "web-server-1",
  "status": "running",
  "region": "us-east-1"
}
```

Parse JSON to work with the data in your automation.

7 PYTHON 'REQUESTS' LIBRARY

The requests library makes API calls simple.

```
import requests
url = "https://api.example.com/v1/servers"
headers = {
  "Authorization": "Bearer YOUR_TOKEN",
  "Accept": "application/json"
}
response = requests.get(url, headers=headers, timeout=10)
print(response.status_code)
print(response.json())
```

Install:

```
pip install requests
```

8 TIMEOUTS

Always set timeouts so your script does not hang forever.

```
response = requests.get(url, timeout=10)
```

Use connect and read timeouts for better control.

```
response = requests.get(url, timeout=(5, 15))
# (connect_timeout, read_timeout)
```

9 HANDLING FAILED REQUESTS

Always check status and handle errors.

```
try:
  response = requests.get(url, timeout=10)
  response.raise_for_status() # Raises HTTPError for 4xx/5xx
  data = response.json()
except requests.exceptions.Timeout:
  print("Request timed out")
except requests.exceptions.HTTPError as e:
  print(f"HTTP error: (e.response.status_code) - {e.response.text}")
except requests.exceptions.RequestException as e:
  print(f"Request failed: {e}")
```

COMMON ERRORS

- Timeout
- ConnectionError
- HTTPError (4xx/5xx)
- TooManyRedirects
- JSONDecodeError

BEST PRACTICE

Log errors with context (url, status, message) so you can debug fast.

10 EXAMPLE: QUERY AN INFRASTRUCTURE API

Example: Get list of EC2 instances (AWS EC2 API)

```
import requests
import json
url = "https://ec2.us-east-1.amazonaws.com/"
params = {
  "Action": "DescribeInstances",
  "Version": "2016-11-15"
}
response = requests.get(url, params=params, timeout=10)
response.raise_for_status()
data = response.json()
for reservation in data["Reservations"]:
  for instance in reservation["Instances"]:
    print(instance["InstanceId"], instance["State"]["Name"])
```

This returns your running instances and their state.



JUNIOR ENGINEER TIP:

APIs are everywhere in DevOps. Master them and you can automate almost any system, cloud service, or tool.



SECURITY REMINDER:

Never hardcode tokens or keys. Use environment variables or secret managers.

GIT AND GITHUB AUTOMATION

Automate repository tasks using the GitHub REST API

1 AUTOMATING REPOSITORY TASKS

Python + GitHub API helps you manage repositories, issues, pull requests, workflows, and more—without manual clicks.

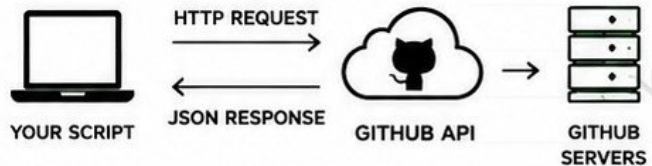
COMMON TASKS YOU CAN AUTOMATE

- ✓ Check repository information
- ✓ Manage issues and pull requests
- ✓ Read workflow runs and status
- ✓ Create labels, branches, and releases
- ✓ Respond to webhooks

2 GITHUB REST API

GitHub exposes a powerful REST API.

Base URL: <https://api.github.com>



3 AUTHENTICATION TOKENS

- Use Personal Access Tokens (Classic) or Fine-Grained Tokens.
- Pass the token in the Authorization header.
- Never hardcode tokens in your script.

```
headers = {
    "Authorization": "Bearer YOUR_TOKEN",
    "Accept": "application/vnd.github+json"
}
```



SECURITY REMINDER

Store tokens in environment variables or secret managers.

4 READING REPOSITORIES

List repositories for the authenticated user.

```
import requests
url = "https://api.github.com/user/repos"
response = requests.get(url, headers=headers)
repos = response.json()

for repo in repos:
    print(f"repo['name']: {repo['html_url']}")
```

You get information about repositories such as name, URL, description, visibility, stars, and more.

5 ISSUES AND PULL REQUESTS

List open issues

```
url = "https://api.github.com/repos/OWNER/REPO/issues"
params = {"state": "open", "per_page": 5}
resp = requests.get(url, headers=headers, params=params)
for issue in resp.json():
    print(f"#{issue['number']} {issue['title']}")
```

List pull requests

```
url = "https://api.github.com/repos/OWNER/REPO/pulls"
params = {"state": "open", "per_page": 5}
resp = requests.get(url, headers=headers, params=params)
for pr in resp.json():
    print(f"PR #{pr['number']} - {pr['title']}")
```

6 WORKFLOW INFORMATION

Check recent workflow runs and their status.

```
url = "https://api.github.com/repos/OWNER/REPO/actions/runs"
params = {"per_page": 5}
resp = requests.get(url, headers=headers, params=params).json()
for run in resp["workflow_runs"]:
    print(f"run['name']: {run['status']} ({run['conclusion']})")
```

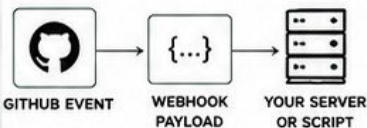
CONCLUSION VALUES

- ✓ success Workflow completed successfully.
- ✗ failure Workflow failed.
- ✗ cancelled Workflow was cancelled.
- 🔄 in_progress Workflow is still running.

7 WEBHOOKS CONCEPT

Webhooks let GitHub notify your system when events happen.

- Events: push, pull_request, issues, workflow_run, release, etc.
- GitHub sends a POST request to your endpoint URL.
- You process the payload and automate action in real time.



8 ENVIRONMENT-BASED CREDENTIALS

Best practice: use environment variables.

```
import os,
token = os.getenv("GITHUB_TOKEN")
if not token:
    raise Exception("GITHUB_TOKEN not set")
headers = {"Authorization": f"Bearer {token}",
    "Accept": "application/vnd.github+json"}
```



BEST PRACTICE

Never commit tokens. Use .env files (ignored by Git) or secret managers.

9 RATE LIMITS

GitHub API has rate limits.

- Authenticated users: 5,000 requests/hour.
- Check remaining limit in response headers.
- Handle rate limit errors (403).

```
resp = requests.get(url, headers=headers)
remaining = resp.headers.get("X-RateLimit-Remaining")
reset = resp.headers.get("X-RateLimit-Reset")
print(f"Remaining: {remaining} | Resets at: {reset}")
```



If remaining is 0, wait until reset time.



JUNIOR ENGINEER TIP

Automate the boring things, so you can focus on solving real engineering problems.

10 PRACTICAL REPOSITORY-STATUS SCRIPT (EXAMPLE)

This script prints a summary of a repository.

```
import os, requests
token = os.getenv("GITHUB_TOKEN")
owner = "OWNER"
repo = "REPO"
headers = {"Authorization": f"Bearer {token}", "Accept": "application/vnd.github+json"}

# Repo info
repo_url = f"https://api.github.com/repos/{owner}/{repo}"
repo = requests.get(repo_url, headers=headers).json()

# Open issues
issues = requests.get(f"{repo_url}/issues", headers=headers, params={"state": "open", "per_page": 1}).json()
open_issues = len([i for i in issues if 'pull_request' not in i])

# Open PRs
prs = requests.get(f"{repo_url}/pulls", headers=headers, params={"state": "open", "per_page": 1}).json()
open_prs = len(prs)

# Recent runs
runs = requests.get(f"{repo_url}/actions/runs", headers=headers, params={"per_page": 1}).json()
workflow = runs["workflow_runs"][0]

print(f"Repo: {repo['full_name']}")
print(f"Owner: {repo['owner']['login']}")
print(f"Repo: {repo['name']}")
print(f"Repo: {repo['description']}")
print(f"Open issues: {open_issues} | Open PRs: {open_prs}")
print(f"Latest workflow: {workflow['name']} - {workflow['conclusion']}")
print(f"Updated at: {workflow['updated_at']}")
print("\n")
```

WHAT THIS SCRIPT DOES

- ✓ Gets repository info
- ✓ Counts open issues
- ✓ Counts open pull requests
- ✓ Checks latest workflow run
- ✓ Prints a quick status report



TRY THIS

Run the script with a real repository and explore more API endpoints.

AWS AUTOMATION WITH BOTO3

AUTOMATE AWS INFRASTRUCTURE SAFELY AND EFFICIENTLY

1 WHY USE SDKS INSTEAD OF MANUALLY CLICKING

- ✓ Speed: Automate repetitive tasks.
- ✓ Consistency: Reduce human errors.
- ✓ Scale: Manage hundreds of resources easily.
- ✓ Integration: Works with CI/CD, scripts, tools.
- ✓ Auditability: Scripted actions are versioned and repeatable.

 **TIP:** Automate what you understand. Start small, then scale.

2 WHAT IS BOTO3?

Boto3 is the official AWS SDK for Python. It lets you interact with AWS services using Python code.



- ★ Supports all AWS services.
- ★ Built on top of botocore.

3 SESSIONS AND CLIENTS

A session represents your connection to AWS. A client is a low-level service interface.

```
import boto3
# Create a session
session = boto3.Session(region_name='us-east-1')
# Create a service client
ec2 = session.client('ec2')
s3 = session.client('s3')
```

- ★ Use clients for low-level operations.
- ★ Use resources for higher-level abstractions.

4 AWS CREDENTIAL CHAIN

Boto3 looks for credentials in this order:

- 1 Environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY)
- 2 Shared credentials file (~/.aws/credentials)
- 3 AWS config file (~/.aws/config)
- 4 Assume Role / Web Identity
- 5 ECS / EKS / EC2 Instance Profile
- 6 AWS SSO / IAM Identity Center

 **NEVER** hardcode AWS keys in your scripts!

5 IAM PERMISSIONS

Boto3 actions require IAM permissions. Example for listing EC2 instances:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances"
      ],
      "Resource": "*"
    }
  ]
}
```

- ✓ Grant least privilege.
- ✓ Only allow what the script needs.

6 LISTING EC2 INSTANCES

Retrieve and display EC2 instances.

```
import boto3
ec2 = boto3.client('ec2')

response = ec2.describe_instances()
for reservation in response['Reservations']:
    for instance in reservation['Instances']:
        print(f"Instance ID: {instance['InstanceId']}")
        print(f"State: {instance['State']['Name']}")
        print(f"Type: {instance['InstanceType']}")
        print(f"{'-' * 40}")
```

Filters Example

```
response = ec2.describe_instances(
    Filters=[{
        'Name': 'instance-state-name',
        'Values': ['running']
    }]
)
```



7 WORKING WITH S3

List buckets and upload a file.

```
import boto3
s3 = boto3.client('s3')

# List all buckets
buckets = s3.list_buckets()
for b in buckets['Buckets']:
    print(b['Name'])

# Upload a file
s3.upload_file('app.log', 'my-bucket', 'logs/app.log')
```



8 RESOURCE TAGGING

Add or update tags on AWS resources.

```
ec2.create_tags(
    Resources=['i-1234567890abcdef0'],
    Tags=[
        {'Key': 'Environment', 'Value': 'Production'},
        {'Key': 'Owner', 'Value': 'DevOps'}
    ]
)
```

- ★ Use consistent tagging for cost tracking, ownership, automation, and cleanup.

9 FILTERING RESOURCES

Filter resources to get only what you need.

```
response = ec2.describe_instances(
    Filters=[
        {'Name': 'tag:Environment', 'Values': ['Prod']},
        {'Name': 'instance-state-name', 'Values': ['running']}
    ]
)
```

- ✓ Filtering reduces data and improves performance and accuracy.

10 ERROR HANDLING

Always handle exceptions in your automation.

```
import boto3
from botocore.exceptions import ClientError
ec2 = boto3.client('ec2')

try:
    response = ec2.describe_instances()
except ClientError as e:
    print(f"Error: {e.response['Error']['Message']}")
    print(f"Code: {e.response['Error']['Code']}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

 **Always fail safely. Log errors with context.**

11 LEAST PRIVILEGE

- ✓ Create dedicated IAM users or roles.
- ✓ Grant only the permissions required.
- ✓ Use IAM roles for EC2, ECS, EKS, Lambda.
- ✓ Regularly review and audit permissions.



12 NEVER EMBED AWS KEYS IN SCRIPTS

- ✗ Don't store keys in code.
- ✗ Don't commit keys to Git.
- ✗ Don't share keys.
- ✗ Don't use long-lived credentials.

-  **USE:**
- ✓ Environment variables
 - ✓ IAM Roles
 - ✓ AWS Secrets Manager
 - ✓ AWS SSO



13 PRACTICAL REPOSITORY-STATUS SCRIPT (EXAMPLE)

This script lists EC2 instances, counts them by state, and prints a summary.

```
import boto3
from collections import Counter
ec2 = boto3.client('ec2')
resp = ec2.describe_instances()
states = []
for r in resp['Reservations']:
    for i in r['Instances']:
        states.append(i['State']['Name'])
summary = Counter(states)
print("EC2 Instance Summary:")
for state, count in summary.items():
    print(f"{state}: {count}")
```

Sample Output

```
EC2 Instance Summary:
running: 8
stopped: 2
pending: 1
shutting-down: 0
terminated: 0
```



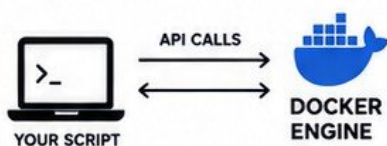
DOCKER AUTOMATION WITH PYTHON

AUTOMATE CONTAINERS AND IMAGES USING THE DOCKER ENGINE API

1 DOCKER ENGINE API

Docker exposes a REST API that lets you manage Docker demons programmatically.

Every Docker CLI command is actually an API call behind the scenes.



- ★ Full control, full automation, zero manual clicks.

2 DOCKER SDK FOR PYTHON

The docker Python SDK is a high-level wrapper around the Docker Engine API.

Install:
pip install docker



- ★ Easier and more Pythonic way to interact with Docker.

3 CONNECTING TO DOCKER

Connect to the Docker Engine (usually via the local Unix socket).

```
import docker

# Connect to local Docker daemon
client = docker.from_env()

# Or connect explicitly
client = docker.DockerClient(
    base_url='unix:///var/run/docker.sock'
)
```

- ✓ From_env() uses environment variables or default Docker socket.

4 LISTING CONTAINERS

List all containers.

```
containers = client.containers.list()
for c in containers:
    print(f"{c.id[:12]}\t{c.name}\t{c.status}")
```

CONTAINER ID	NAME	STATUS
a1b2c3d4e5f6	web-app	running
b2c3d4e5f6a7	redis-cache	exited
c3d4e5f6a7b8	worker	running

>_ Shows ID, name, and current status.

5 INSPECTING CONTAINERS

Get detailed information about a container.

```
container = client.containers.get('web-app')
info = container.attrs
print(info['State']['Status'])
print(info["NetworkSettings"]["IPAddress"])
```

Returns:

- State, config, network, mounts
- IP addresses, ports
- Resource usage and more

🔍 Use inspect to make decisions based on container state.

6 STARTING AND STOPPING CONTAINERS

Start, stop, restart containers.

```
container = client.containers.get('web-app')
container.start()
container.stop()
container.restart()
print(container.status)
```



START



STOP



RESTART

- ★ Always check status after actions.

7 READING LOGS

Stream or fetch container logs.

```
container = client.containers.get('web-app')
logs = container.logs(tail=50, timestamps=True)
print(logs.decode('utf-8'))
```

Stream logs in real time:

```
for line in container.logs(stream=True):
    print(line.decode('utf-8'), end='')
```

>_ Great for debugging and monitoring.

8 WORKING WITH IMAGES

List, pull, and inspect images.

```
images = client.images.list()
for img in images:
    print(f"{img.id[:12]}\t{img.tags}")
```

```
client.images.pull('nginx:latest')
image = client.images.get('nginx:latest')
print(image.attrs['Size'])
```

 Pull, remove, and inspect images programmatically.

9 CONTAINER HEALTH CHECKS

Check container health status.

```
container = client.containers.get('web-app')
status = container.attrs['State']['Health']
if status:
    print("Health Status:", status['Status'])
    print("Last Check:", status['Last'])
else:
    print("No health check configured")
```

💓 Health checks help ensure your containers are actually healthy.

10 DOCKER SOCKET SECURITY WARNING



THE DOCKER SOCKET GIVES ROOT-LEVEL ACCESS TO YOUR HOST.

Anyone who can access /var/run/docker.sock can:

- Start privileged containers
- Mount the host filesystem
- Access host processes
- Escalate to root on the host

BEST PRACTICES

- Do NOT expose the Docker socket to untrusted users.
- Use a proxy or restricted API when possible.
- Run automation with least privilege.
- Never use the socket in codes that others can run.



SAFETY TIP

Run automation with the minimum permissions required. Never give more access than needed.

```
# GOOD PRACTICE
# Use a non-root user and limit access
# to the Docker socket
useradd -m docker-bot
usermod -aG docker docker-bot
```



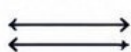
KUBERNETES AUTOMATION WITH PYTHON

AUTOMATE CLUSTERS AND WORKLOADS USING THE KUBERNETES PYTHON CLIENT

1 KUBERNETES PYTHON CLIENT

The official Kubernetes Python client lets you interact with the Kubernetes API.

Install:
pip install kubernetes



YOUR SCRIPT

KUBERNETES API

★ Everything you can do with kubectl, you can automate with Python.

2 KUBECONFIG

Use your kubeconfig file to connect from your machine (outside the cluster).

```
from kubernetes import config, client

# Load kubeconfig from default location
config.load_kube_config()

v1 = client.CoreV1Api()
print("Connected to cluster:",
v1.get_api_versions().versions[0].git_version)
```

✓ Kubeconfig contains clusters, users, and contexts.

3 IN-CLUSTER AUTHENTICATION

When your script runs inside a Pod, use the in-cluster service account.

```
from kubernetes import config, client

# This loads service account token
# and CA cert from the Pod
config.load_incluster_config()

v1 = client.CoreV1Api()
print("In-cluster auth successful")
```

✓ No kubeconfig needed inside the cluster. Uses service account automatically.

4 LISTING PODS

List all Pods in a namespace.

```
from kubernetes import client, config
config.load_kube_config()

v1 = client.CoreV1Api()
pods = v1.list_namespaced_pod(namespace="default")
for pod in pods.items:
    print(f"{pod.metadata.name}\t{pod.status.phase}")
```

POD STATUS PHASES

- Pending Pod accepted by cluster
- Running Pod is running
- Succeeded All containers completed successfully
- Failed At least one container failed
- Unknown Status cannot be determined

🔍 Quick overview of what is running.

5 INSPECTING DEPLOYMENTS

Get details about a Deployment.

```
from kubernetes import client, config
config.load_kube_config()

apps_v1 = client.AppsV1Api()
dep = apps_v1.read_namespaced_deployment(
    name="web-app", namespace="default")
print("Replicas:", dep.status.replicas)
print("Available:", dep.status.available_replicas)
print("Image:", dep.spec.template.spec.containers[0].image)
```

🔍 Know your desired vs available state, replicas, strategy, and more.

6 READING POD STATUS

Inspect detailed status of a Pod.

```
from kubernetes import client, config
config.load_kube_config()

v1 = client.CoreV1Api()
pod = v1.read_namespaced_pod(
    name="web-app-7d9f6c8d9-abcde",
    namespace="default")
print("Phase:", pod.status.phase)
print("Node:", pod.spec.node_name)
for cs in pod.status.container_statuses:
    print(cs.name, cs.ready, cs.restart_count, cs.state)
```

🔍 Get container states, restarts, IP, node and conditions.

7 READING LOGS

Stream or fetch Pod logs.

```
from kubernetes import client, config
config.load_kube_config()

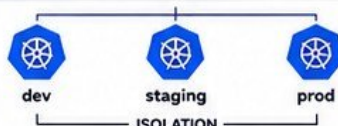
v1 = client.CoreV1Api()
logs = v1.read_namespaced_pod_log(
    name="web-app-7d9f6c8d9-abcde",
    namespace="default",
    container="web",
    tail_lines=100,
    timestamps=True)
print(logs)
```

📄 Great for debugging and monitoring.

8 NAMESPACES

Work across namespaces.

```
v1 = client.CoreV1Api()
ns_list = v1.list_namespace()
for ns in ns_list.items:
    print(ns.metadata.name)
```



Namespaces separate environments and teams.

9 WATCHING RESOURCES

Watch for changes to resources in real time.

```
from kubernetes import watch, client, config
config.load_kube_config()

v1 = client.CoreV1Api()
w = watch.Watch()
for event in w.stream(v1.list_namespaced_pod,
    namespace="default", timeout_seconds=60):
    print(event['type'], event['object'].metadata.name,
    event['object'].status.phase)
```

👁 Useful for building controllers and real-time automation.

10 RBAC (ROLE-BASED ACCESS CONTROL)

Automation users (service accounts) need permissions to do things in the cluster.

EXAMPLE ROLE (READ PODS)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-pods
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
```

EXAMPLE BINDING

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: automation-bot
  namespace: default
roleRef:
  kind: Role
  name: read-pods
  apiGroup: rbac.authorization.k8s.io
```

🔒 RBAC controls WHAT actions identities can perform and WHERE.

11 WHY AUTOMATION IDENTITIES NEED LEAST PRIVILEGE



AUTOMATION
IDENTITY



PERMISSION
GRANTED



ONLY WHAT IS
NEEDED



LIMIT DAMAGE
IF COMPROMISED

FOLLOW LEAST PRIVILEGE

- Grant minimum permissions required to complete the task.
- Avoid cluster-admin for automation.
- Review permissions regularly.
- Use separate identities per purpose/environment.

OVER-PERMISSION IS RISKY!

- Bigger blast radius
- Easier lateral movement
- Harder to audit and review



JUNIOR ENGINEER TIP

Understand Kubernetes concepts first.
Automation is just code speaking to the API.



SECURITY REMINDER

Protect your kubeconfig and tokens.
Never commit them to Git.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SSH AND REMOTE SERVER AUTOMATION

AUTOMATE LINUX SERVERS SAFELY AND RELIABLY WITH PYTHON AND PARAMIKO

1 WHEN REMOTE AUTOMATION IS APPROPRIATE

- ✓ When you need to run commands on many servers.
- ✓ When no API is available.
- ✓ When you must manage legacy systems.
- ✓ When configuration changes are required.
- ✓ When collecting logs or system information.

★ Use SSH automation for tasks that SSH access can do well and safely.

2 PARAMIKO CONCEPT

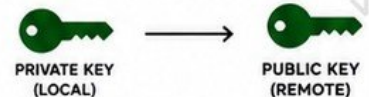
Paramiko is a Python library that implements the SSHv2 protocol.



★ Paramiko lets you connect and automate servers programmatically.

3 SSH KEYS

- Use SSH key pairs for secure authentication.
- Avoid passwords.
- Private key stays on your local machine.
- Public key is placed on the remote server (~/.ssh/authorized_keys).



🛡️ Keys provide strong, non-interactive authentication.

4 CONNECTING TO HOSTS

Connect to a remote host using Paramiko.

```
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(
    paramiko.RejectPolicy())
client.connect(
    hostname='101.0.0.10',
    username='ec2-user',
    key_filename='/home/user/.ssh/id_rsa',
    port=22,
    timeout=10
)
print('Connected!')
```

🔍 Always set host key policy and timeouts.

5 EXECUTING REMOTE COMMANDS

Run commands on the remote server.

```
stdin, stdout, stderr = client.exec_command('uptime')
output = stdout.read().decode()
error = stderr.read().decode()
exit_status = stdout.channel.recv_exit_status()

print('OUTPUT:\n', output)
print('ERROR:\n', error)
print('EXIT STATUS:', exit_status)
```

➤ `exec_command()` returns `stdin`, `stdout`, and `stderr`.

6 CAPTURING OUTPUT

Read and process command output.

```
stdin, stdout, stderr = client.exec_command('df -h')
output = stdout.read().decode().strip()
error = stderr.read().decode().strip()

if output:
    print(output)
if error:
    print('ERROR:', error)
```

📄 Always capture both `stdout` and `stderr` to handle failures.

7 TIMEOUTS

Timeouts prevent scripts from hanging forever.

```
client.connect(
    hostname='10.0.0.10',
    username='ec2-user',
    key_filename='~/.ssh/id_rsa',
    timeout=10, # Connection timeout (seconds)
    banner_timeout=10 # SSH banner timeout
)
```

🕒 Set both connection and banner timeouts.

8 HANDLING UNREACHABLE HOSTS

Always handle connection and command errors.

```
try:
    client.connect(hostname=host, username=user,
                  key_filename=key, timeout=10)
except paramiko.ssh_exception.NoValidConnectionsError:
    print(f'[{host}] Host unreachable')
    return
except Exception as e:
    print(f'[{host}] Connection error: {e}')
    return
```

⚠️ Do not let one bad host stop your automation.

9 MULTIPLE-SERVER AUTOMATION

Loop through a list of hosts.

```
hosts = ['10.0.0.10', '10.0.0.11', '10.0.0.12']
user = 'ec2-user'
key = '/home/user/.ssh/id_rsa'
for host in hosts:
    try:
        client = paramiko.SSHClient()
        client.set_missing_host_key_policy(
            paramiko.RejectPolicy())
        client.connect(hostname=host, username=user,
                      key_filename=key, timeout=10)
        stdin, stdout, stderr = client.exec_command('hostname')
        print(f'[{host}] -> {stdout.read().decode().strip()}')
        client.close()
    except Exception as e:
        print(f'[{host}] ERROR: {e}')
```

👥 Automate at scale, safely and consistently.

10 HOST-KEY VERIFICATION

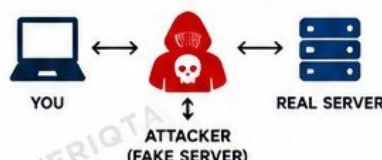
Verify host keys to confirm you are connecting to the correct server.

```
client = paramiko.SSHClient()
client.set_missing_host_key_policy(
    paramiko.RejectPolicy()) # REJECT unknown hosts
# Alternative: load known_hosts
client.load_system_host_keys()
```

✓ `RejectPolicy()` is the safest default.

11 WHY BLINDLY TRUSTING HOSTS IS DANGEROUS

MAN-IN-THE-MIDDLE (MITM) ATTACK



- ✗ An attacker can impersonate a server.
- ✗ You may send credentials to the wrong host.
- ✗ Commands and data can be intercepted.
- ✗ Infrastructure can be compromised.

🛡️ Always verify host keys. Never blindly trust unknown hosts.

12 BEST PRACTICES



Use SSH keys, not passwords.



Verify host keys. Reject unknown hosts.



Use timeouts for all connections and commands.



Handle errors and unreachable hosts gracefully.



Log all actions for audit and troubleshooting.



Principle of least privilege for remote users.

PYTHON IN CI/CD PIPELINES

AUTOMATE • VALIDATE • DEPLOY • VERIFY

1 WHERE PYTHON RUNS IN PIPELINES

Python scripts add logic, intelligence, and safety to every stage of your delivery pipeline.



2 COMMON USES OF PYTHON IN CI/CD

- **Validation Scripts** – Validate inputs, versions, or policies
- **Configuration Checks** – Ensure configs are correct
- **Deployment Gates** – Approve or block deployments
- **API Calls** – Interact with services, clouds, Git, etc.
- **Smoke Tests** – Verify app or service is running
- **Data Processing** – Parse, transform, and validate data
- **Reporting** – Generate build, test, and deployment reports

3 EXIT STATUS AND PIPELINE FAILURE

Python exits with a status code. CI/CD uses it to decide success or failure.

EXIT CODE	MEANING
0	SUCCESS
1	GENERAL ERROR
2	INVALID INPUT
3+	CUSTOM ERROR CODES

Example:

```
import sys

def main():
    ok = False
    if ok:
        print("All good!")
        sys.exit(0)
    else:
        print("Validation failed!")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

Non-zero exit code = Pipeline fails.

4 POPULAR CI/CD PLATFORMS



GITHUB ACTIONS

Use Python in workflows with steps and jobs.



GITLAB CI

Use Python in .gitlab-ci.yml jobs and stages.



JENKINS

Run Python in pipeline scripts or shell steps.

5 EXAMPLE: PYTHON VALIDATION SCRIPT

```
import os, json, sys

required = ["APP_VERSION", "ENVIRONMENT"]
missing = [key for key in required if not os.getenv(key)]

if missing:
    print(f"Missing variables: {', '.join(missing)}")
    sys.exit(1)

print("All required variables found")
sys.exit(0)
```

What it does:

- ✓ Checks required env vars
- ✓ Fails the pipeline if any are missing
- ✓ Prevents bad deployments
- ✓ Stops early, saves time

6 PASSING CONFIGURATION SAFELY

- ✓ Use Environment Variables
Do not hardcode values.
- ✓ Use Config Files
Store configs in repo or artifact store.
- ✓ Use Secrets Managers
Vault, AWS Secrets Manager, etc.
- ✓ Use Encrypted Secrets in CI/CD
GitHub Secrets, GitLab CI Variables, Jenkins Credentials.



7 EXAMPLE: PIPELINE SNIPPETS



GITHUB ACTIONS

```
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - name: Run Validation
        run: python scripts/validate.py
      - name: Run Tests
        run: python -m pytest -q
```



GITLAB CI

```
stages:
  - validate
  - test
  - deploy

validate:
  stage: validate
  image: python:3.11
  script:
    - pip install -r requirements.txt
    - python scripts/validate.py

test:
  stage: test
  script:
    - python -m pytest -q
```



JENKINS (Declarative Pipeline)

```
pipeline {
  agent any
  stages {
    stage('Validate') {
      steps {
        sh 'python scripts/validate.py'
      }
    }
    stage('Test') {
      steps {
        sh 'python -m pytest -q'
      }
    }
  }
}
```

8 KEEPING SECRETS OUT OF LOGS

NEVER DO THIS

```
print(f"API Key: {os.getenv('API_KEY')}")
print(config) # Might contain secrets
```

DO THIS INSTEAD

- Mask secrets in logs.
- Never print tokens, keys, or passwords.
- Use built-in masking (GitHub, GitLab, Jenkins).
- Log only what you must.
- Use short-lived credentials.



JUNIOR ENGINEER TIP

Treat your pipeline like production. If secrets leak in CI/CD, attackers will find them.



GOOD AUTOMATION ISN'T JUST ABOUT MAKING THINGS WORK.
IT'S ABOUT MAKING THEM WORK SAFELY, REPEATABLY, AND VERIFIABLY.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

MONITORING AND INCIDENT AUTOMATION

DETECT • DIAGNOSE • ALERT • RESPOND

1 AUTOMATED HEALTH CHECKS

Continuously check the health of systems, services, and applications.



SERVERS



SERVICES

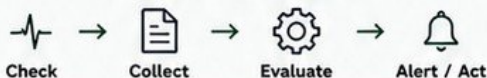


API
ENDPOINTS



DATABASES

EXAMPLE HEALTH CHECK FLOW



2 METRICS

Collect quantitative data over time.

- CPU Usage
- Memory Usage
- Disk Usage
- Network I/O
- Request Rate
- Error Rate
- Response Time

TOOLS EXAMPLES

Prometheus • CloudWatch
Datadog • Grafana

3 LOGS

Logs explain what happened. Collect, centralize, and search logs.



ACCESS
LOGS



APPLICATION
LOGS



SYSTEM
LOGS

BEST PRACTICE

Centralize logs and keep timestamps in UTC.

4 API HEALTH ENDPOINTS

Expose an endpoint that returns system health.
Example: /health

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/health")
def health():
    return jsonify({"status": "ok"}, 200)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

COMMON RESPONSES

200 OK
503 UNHEALTHY
500 ERROR

5 THRESHOLDS

Define limits. Trigger alerts when breached.

METRIC	THRESHOLD EXAMPLE
CPU Usage	> 80% for 5m
Memory Usage	> 85% for 5m
Disk Usage	> 85%
Error Rate	> 5%
Response Time	> 2s



SET SMART THRESHOLDS

Avoid noisy alerts.
Alert on impact, not noise.

6 DETECTING DISK-SPACE PROBLEMS

Python example using psutil.

```
import psutil

def check_disk(threshold=10):
    usage = psutil.disk_usage('/')
    used_percent = usage.percent
    free_gb = usage.free / (1024**3)
    if used_percent > (100 - threshold):
        return {"status": "critical",
                "used_percent": used_percent,
                "free_gb": round(free_gb, 2)}
    return {"status": "ok",
            "used_percent": used_percent}
```

- Alert when free space < threshold.
- Include used %, free GB in alert.

7 DETECTING FAILED SERVICES

Check service/process status.
Example with systemctl.

```
import subprocess

def check_service(name):
    result = subprocess.run(
        ["systemctl", "is-active", name],
        capture_output=True, text=True)
    if result.stdout.strip() == "active":
        return {"status": "ok"}
    return {"status": "critical",
            "service": name,
            "message": result.stdout.strip()}
```

CHECK PATTERNS

- ✓ systemctl status
- ✓ Docker container status
- ✓ Kubernetes Pod status
- ✓ Application port checks

8 ALERT ENRICHMENT

Add useful context to every alert.



Host / Service



Current Status



Metric Value



Threshold



Time / Duration



Runbook Link

WHY IT MATTERS

Engineers act faster with context, not raw alerts.

9 AUTOMATED DIAGNOSTICS

Collect data automatically when an issue is detected.

Examples:

- ✓ Collect last 100 log lines
- ✓ Capture system metrics
- ✓ Take process snapshots
- ✓ Check disk, memory, CPU
- ✓ Network connectivity test
- ✓ Store diagnostics to bucket
- ✓ Attach diagnostics to alert



TIP

Diagnostics reduce MTTR and guesswork.

10 NOTIFICATIONS

Send alerts to the right people through the right channels.



Email



Slack / Teams



PagerDuty / Opsgenie



SMS / Phone



Webhook / API



BEST PRACTICE

Route alerts based on severity and time.

11 WHY AUTOMATIC REMEDIATION NEEDS GUARDRAILS



RISK OF DAMAGE

Bad automation can make things worse.



USE SAFETY CONTROLS

- Dry-run first
- Confirm before action
- Limit blast radius



VERIFY BEFORE & AFTER

Always verify the state before and after taking any action.



HAVE ESCAPE HATCHES

Provide rollback or automatic revert options.



HUMAN APPROVAL

Require approval for high-risk actions.



JUNIOR ENGINEER TIP

Monitor everything. Automate carefully. Observe, understand, then automate.



AUTOMATE WITH PURPOSE AND SAFETY.

Faster detection. Smarter response. More reliable systems.

WRITING PRODUCTION-SAFE DEVOPS AUTOMATION

BUILD AUTOMATION THAT IS RELIABLE, SAFE, AND TRUSTWORTHY

1 IDEMPOTENCY



Make your automation safe to run multiple times. Same input = same result.

EXAMPLE

```
# Check before creating a resource
if not resource_exists(name):
    create_resource(name)
else:
    log("Resource already exists")
```

2 DRY-RUN MODE



Simulate actions without making changes. Preview what would happen.

EXAMPLE

```
$ python deploy.py --dry-run
[DRY-RUN] Would create 3 resources
[DRY-RUN] Would update security group
[DRY-RUN] No changes made
```

3 INPUT VALIDATION



Validate all inputs early. Fail fast with clear error messages.

CHECK

- Required fields present
- Correct data types
- Acceptable value ranges
- Safe characters only

4 TIMEOUTS



Always set timeouts for network calls and commands to avoid hanging forever.

EXAMPLE

```
response = requests.get(url, timeout=10)
subprocess.run(cmd, timeout=30)
```

5 RETRIES



Retry transient failures automatically. But don't retry everything.

EXAMPLE

```
for attempt in range(3):
    try:
        call_api()
        break
    except TemporaryError:
        log("Retrying...")
```

6 EXPONENTIAL BACKOFF



Increase wait time between retries to reduce load and thundering herd.

EXAMPLE

```
wait = 1
for attempt in range(5):
    try:
        call_api()
        break
    except Exception:
        time.sleep(wait)
        wait *= 2 # 1, 2, 4, 8, 16...
```

7 RATE LIMITS



Respect API and service rate limits. Avoid being throttled or blocked.

BEST PRACTICES

- Read API docs for limits
- Use tokens / buckets
- Add delays between calls
- Handle HTTP 429 responses

8 STRUCTURED LOGGING



Use structured logs for better search, alerts, and troubleshooting.

EXAMPLE (JSON LOG)

```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "level": "INFO",
  "event": "deploy_started",
  "service": "web-api",
  "environment": "production",
  "user": "ci-bot"
}
```

9 SECRET HANDLING



Never hardcode secrets. Store and access them securely.

BEST PRACTICES

- Use Secrets Managers (AWS, Vault)
- Use environment variables
- Never print secrets in logs
- Mask secrets in CI/CD outputs

10 LEAST PRIVILEGE



Use the minimum permissions required to do the job.

BEST PRACTICES

- Least-privilege IAM roles
- Scoped API tokens
- Separate identities per task
- Regularly review permissions

11 TESTING



Test your automation like production software.

TEST TYPES

- Unit tests - test functions
- Integration tests - test APIs
- Mock external services
- Test happy path and failures

12 AUDITABILITY



Every action should be traceable and reviewable.

BEST PRACTICES

- Log who/what/when/where
- Store logs centrally
- Include request IDs
- Keep audit logs immutable

13 ROLLBACK



Have a plan to revert changes when something goes wrong.

BEST PRACTICES

- Version everything
- Snapshot before changes
- Automated rollback steps
- Validate rollback success

14 PREVENT DESTRUCTIVE AUTOMATION



Protect systems and data from accidental or malicious destructive actions.

SAFETY CONTROLS

- Confirmation prompts
- Resource allowlists / denylists
- Environment protections
- Multi-step safeguards

15 HUMAN APPROVAL FOR HIGH-RISK OPS



Require human sign-off before performing high-impact actions.

BEST PRACTICES

- Approval workflows in CI/CD
- ChatOps approvals (Slack, Teams)
- Change management integration
- Record approvals for audit



CORE PRINCIPLE

AUTOMATION SHOULD MAKE OPERATIONS SAFER, NOT RISKIER. BUILD WITH DEFENSE IN DEPTH AND ASSUME FAILURE WILL HAPPEN.



JUNIOR ENGINEER TIP

AUTOMATE GRADUALLY. START SMALL, TEST THOROUGHLY, AND EARN TRUST.

FINAL PRODUCTION LAB

BUILD A DEVOPS AUTOMATION TOOL

BUILD • CHECK • AUTOMATE • OBSERVE • RECOVER

1 LAB OBJECTIVE

Build a Python automation tool that inspects your infrastructure, detects problems, reports them clearly, and helps you act safely. This is your capstone: everything you've learned so far comes together.

2 WHAT YOUR AUTOMATION TOOL MUST DO

- 1 Read configuration from YAML 
- 2 Load credentials securely 
- 3 Query infrastructure 
- 4 Check multiple servers or workloads 
- 5 Inspect CPU, memory, disk, and service health 
- 6 Call an HTTP API 
- 7 Detect unhealthy resources 
- 8 Produce structured logs 
- 9 Handle API and network failures 
- 10 Retry temporary failures 
- 11 Generate a health report 
- 12 Return meaningful exit codes 
- 13 Run the tool from CI/CD 
- 14 Introduce a deliberate failure 
- 15 Diagnose it 
- 16 Verify recovery 

4 EXAMPLE HEALTH REPORT (JSON)

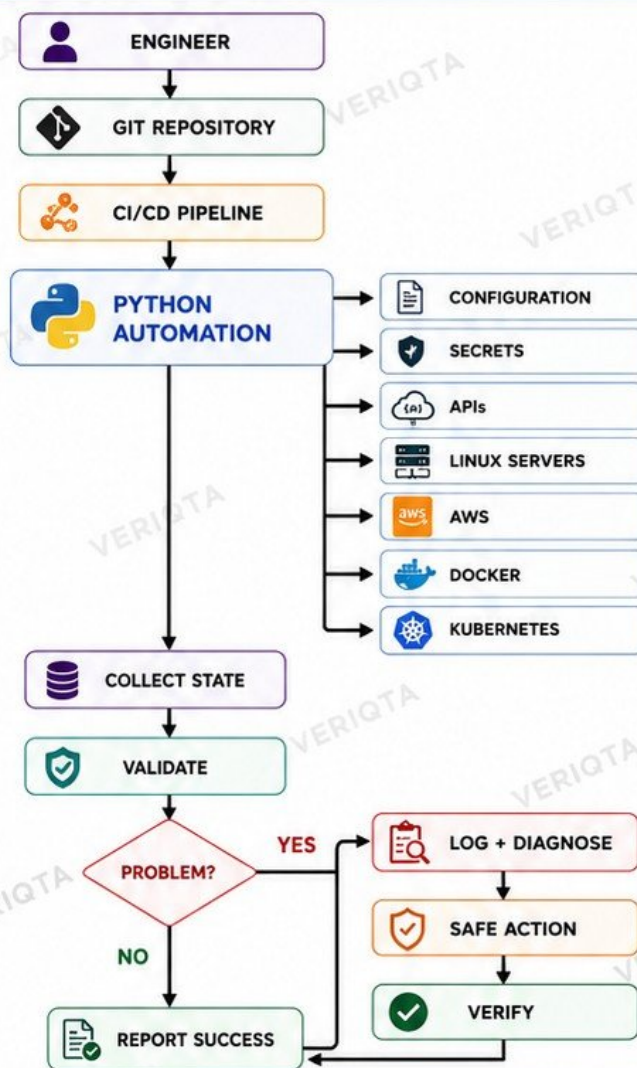
```
{
  "timestamp": "2025-05-20T10:30:00Z",
  "overall_status": "unhealthy",
  "summary": {"total": 5, "healthy": 3, "unhealthy": 2},
  "checks": [
    {
      "name": "web-01",
      "status": "healthy",
      "cpu": 23,
      "memory": 45,
      "disk": 60
    },
    {
      "name": "db-01",
      "status": "unhealthy",
      "cpu": 85,
      "memory": 91,
      "disk": 92
    }
  ],
  "errors": ["Timeout while calling API /v1/metrics"]
}
```



JUNIOR ENGINEER TIP

Automate with purpose. Validate everything.
Observe constantly. Fail safely. Improve continuously.

3 FINAL ARCHITECTURE



5 PROGRESSION YOU HAVE FOLLOWED



REMEMBER

Automation without guardrails is dangerous.
Add checks, approvals, and safe defaults.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta